

Assignment Five

Property-Based Testing

Due: 2026-10-11

Synopsis: Designing property-based tests for the APL printer, parser, evaluator and checker: a generator with measured coverage, a round-trip property, and a property that exposes a faulty assumption

1 Introduction

In this assignment you will practise the two halves of property-based testing: stating properties, and designing test data with high falsification power.

You will work on the *pure* APL components you built in earlier assignments: the printer, the parser, the evaluator and the checker. You will build a generator for expressions whose distribution is good enough to satisfy a given *coverage specification*, and then use it to test two properties—one that should hold, and one that does not.

Chapter 5 of the course notes covers everything you need. In particular, `cover`, `checkCoverage` and `tabulate` are discussed in the section on test data design.

2 Task: A generator with measured coverage

The handout contains a generator `genExp` for APL expressions that is correct but useless: it produces expressions whose distribution barely exercises any interesting cases. The handout also contains a property `expCoverage`, which states, as a machine-checkable coverage specification, what the distribution ought to look like.

Change the definition of `genExp` so that `expCoverage` passes. In particular this means that

- Between 20% and 80% of expressions should have domain errors (division by zero or negative exponent) reported by `checkExp`.
- Between 20% and 80% of expressions should have type errors reported by `checkExp`.
- Between 5% and 30% of expressions should have variable errors reported by `checkExp`.
- At least 50% of expressions should contain a variable between 2 and 4 characters long.

You *are* permitted to change the signature of `genExp`, but make sure to also change the definition of `arbitrary` for `Exp` accordingly. You must not weaken `expCoverage`.

2.1 Suggested implementation

Use `frequency` instead of `oneof` to control the distribution, and keep track of the variables in scope by adding an additional parameter of type `[VName]` to `genExp`. Use `sample` and `tabulate` while developing the generator: guessing at weights and rerunning `checkCoverage` is much slower than looking at what you are producing.

3 Task: A round-trip property for printing and parsing

Implement the property `parsePrinted`, which checks that printing an expression and parsing the resulting string gives back the original expression.

When `QuickCheck` finds a counterexample to `parsePrinted` you must decide whether the error is in the *implementation*, the *generator* or the *property*. Then fix the error and continue until the property passes. You will need to fix errors of more than one of these three kinds.

Note that the handout already asks for 10000 tests per property, via `withMaxSuccess` in `properties`. **Do not reduce this number:** some of the properties below fail only rarely, and the number of tests is part of what makes your test suite reproducible for us.

4 Task: A property for checking and evaluating

Implement the property `onlyCheckedErrors`, which checks that evaluating an expression produces only errors that `checkExp` predicted.

`checkExp` returns a *list* of errors. The intention is that this list contains every error that evaluation of the expression *might* produce. So define `onlyCheckedErrors` to evaluate the given expression and, if evaluation returns an error, to check that this error occurs in the list returned by `checkExp`.

This property does *not* hold: `checkExp` rests on an assumption that is not justified. You are *not* asked to repair `checkExp`. You are asked to find a counterexample by testing, and then to explain in your report what the faulty assumption is (see the report questions below).

5 Your Report

You are expected to describe your test design and *interesting* aspects of your implementation. Do *not* give a line-by-line walkthrough of your code—relevant details of code that is not self-explanatory (the ideal) should be documented compactly in the source code. Most importantly, reflect on the *quality* of your code:

- Why is it functionally correct? (Is it?) Provide arguments addressed to a skeptical reader, not yourself.
- Is there an improvement you would have liked to make, but didn't include?

It is more important to reflect on the strengths or shortcomings of your solution than it is to have a complete, but unexplained solution.

5.1 The structure of your report

Your report must be structured as follows:

Introduction: Briefly mention general concerns, any ambiguities in the problem text and how you resolved them, and your own estimation

of the quality of your solution. Briefly mention whether your solution is functional, which test cases cover its functionality, which test cases it fails for (if any), and what you think might be wrong.

A section answering the following numbered questions:

1. Can programs produced by your `genExp` loop infinitely when evaluated? If so, is it possible to avoid this?
2. Which of the four coverage requirements in `expCoverage` was hardest to satisfy, and what did you do to satisfy it?
3. Which counterexamples did `parsePrinted` produce? For each counterexample, which component (implementation, generator or property) did you fix, and why was that the right place to fix it?
4. Which assumption underlying `checkExp` is not justified, and which counterexample did `onlyCheckedErrors` produce to demonstrate this?
5. Your generator satisfies `expCoverage`. Describe a class of expressions that your generator will nevertheless produce essentially never, but that would be worth testing. How would you change either the generator or `expCoverage` to cover it?

All else being equal, **a short report is a good report.**

6 Code handout

The code handout consists of the following nontrivial files.

- `a5.cabal`: Cabal build file. **Do not modify this file.**
- `runtests.hs`: Test execution program. **Do not modify this file.**
- `apl.hs`: A command line interpreter for APL. You will not need it, but it may be convenient for trying out expressions by hand.
- `src/APL/AST.hs`: The APL AST definition. **You may only modify `printExp` in this file.**

- `src/APL/Error.hs`: The APL Error definition. **Do not modify this file.**
- `src/APL/Parser.hs`: The parser.
- `src/APL/Check.hs`: The checker.
- `src/APL/Eval.hs`: The APL evaluation function.
- `src/APL/Tests.hs`: The property tests. This is where most of your work goes.

7 Deliverables for This Assignment

You must submit the following items:

- A single PDF file, A4 size, no more than 5 pages, structured as specified in the report section above.
- A single zip/tar.gz file with all code relevant to the implementation, including at least all the files from the handout. For this assignment it is not necessary to add additional files.

Remember to follow the general assignment rules listed on the course homepage.

8 Assessment

You will get written qualitative feedback, and points from zero to four. There are no resubmissions, so please hand in what you managed to do, even if you have not solved the assignment completely.